

Apprentissage automatique

Classification - K plus proches voisins (KNN)

Cours : Séance 3

Université Assane Seck de Zigunchor
Master Génie logiciel et Réseau sécurité

11 novembre 2025

Objectif général

- Appliquer et évaluer une classification par les plus proches voisins sur un jeu de données.

Objectifs spécifiques

- 1 Appliquer une classification par les k plus proches voisins.
- 2 Choisir un paramètre k optimal pour une classification par KNN.
- 3 Choisir une métrique adaptée pour une classification par KNN.
- 4 Évaluer une classification par KNN.
- 5 Sélectionner et évaluer une méthode de classification.

- 1 Généralités sur la classification
- 2 K plus proches voisins (KNN)

- Étude de la relation entre une variable Y catégorielle à m modalités :

$$Y = (y_1, \dots, y_n)^t, \quad y_i \in \{M_1, \dots, M_m\}$$

- Et p variables explicatives supposées dans $\mathbb{R}$:

$$X_1, \dots, X_p \text{ avec } X_i = (x_{1i}, \dots, x_{ni})$$

- Classification binaire : $m = 2$
- Classification multi-classes : $m > 2$

Applications de la classification

- **Systèmes de recommandations :**
 - Recommandation de films (vidéo à la demande)
 - Recommandation d'amis (réseaux sociaux)
 - Recommandation de produits (grande distribution)
- **Reconnaissance de formes :**
 - Reconnaissance de texte manuscrit
 - Reconnaissance de visages
- **Diagnostic médical :**
 - Évaluation des risques de cancer
 - Évaluation des risques de diabète
- Classification d'e-mails (spam / non spam)

Exemple

Weather [Quinlan86]

Attributs	Pif	Temp	Humid	Vent
Valeurs possibles	soleil,couvert,pluie	chaud,bon,frais	normale,haute	vrai,faux

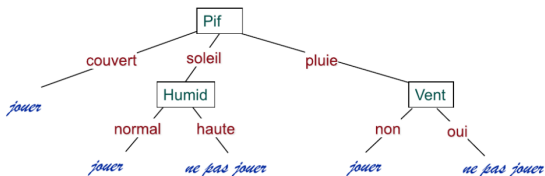
N°	Pif	Temp	Humid	Vent	Golf
1	soleil	chaud	haute	faux	NePasJouer
2	soleil	chaud	haute	vrai	NePasJouer
3	couvert	chaud	haute	faux	Jouer
4	pluie	bon	haute	faux	Jouer
5	pluie	frais	normale	faux	Jouer
6	pluie	frais	normale	vrai	NePasJouer
7	couvert	frais	normale	vrai	Jouer
8	soleil	bon	haute	faux	NePasJouer
9	soleil	frais	normale	faux	Jouer
10	pluie	bon	normale	faux	Jouer
11	soleil	bon	normale	vrai	Jouer
12	couvert	bon	haute	vrai	Jouer
13	couvert	chaud	normale	faux	Jouer
14	pluie	bon	haute	vrai	NePasJouer

la classe

(Jeu de données illustratif pour la classification)

Exemple : Arbre de décision

- Exemple de classification basée sur un arbre de décision.
- Arbre final obtenu :



Source : <http://cedric.cnam.fr/vertigo/cours/ml2/coursArbresDecision.html#bf84>

Les principales méthodes de classification

- K plus proches voisins (k-NN)
- Arbres de décision
- Réseaux de neurones
- Machines à vecteurs de support (SVM)
- Régression logistique
- Forêts aléatoires

Critères de choix et d'évaluation

- **Types de données** : discrètes ou continues.
- **Complexité** : rapidité de l'algorithme / occupation mémoire.
- **Robustesse** : résistance aux valeurs extrêmes, valeurs manquantes, doublons, etc.
- **Scalabilité** : capacité à gérer la montée en charge.
- **Capacité de généralisation** : aptitude à bien prédire sur de nouvelles données.

K plus proches voisins (KNN)

Introduction

- Algorithme de prédiction non paramétrique, simple mais puissant.
- Permet de construire des frontières de décision complexes.
- Principe : « **qui se ressemble s'assemble** ».
- Utilise les étiquettes des exemples les plus proches pour prédire une classe.

Objectifs du chapitre

- 1 Implémenter l'algorithme des k plus proches voisins.
- 2 Calculer distances et similarités pour divers types de données.
- 3 Définir la frontière de décision de l'algorithme.
- 4 Comprendre ses limites en haute dimension.

Principe de la méthode

- Les objets similaires sont proches dans l'espace des caractéristiques.
- Pour classer une observation x :
 - 1 Identifier les k échantillons d'apprentissage les plus proches selon une distance $d(x, x_i)$.
 - 2 Attribuer à x la classe majoritaire de ces k voisins.

Diagramme de Voronoï

- Partitionne l'espace en cellules où chaque point est plus proche d'un centre donné que de tout autre.
- Permet de visualiser la frontière de décision du modèle.

$$X = \bigcup_y \left\{ x \in X \mid \arg \min_{i=1, \dots, n} d(x, x_i) = y \right\}$$



Exemple : 25 points en deux dimensions.

Principe de la méthode k-NN

- La méthode des k plus proches voisins (**k-NN**) classe un échantillon selon ses k voisins les plus proches.
- Si $\{x_{i1}, x_{i2}, \dots, x_{ik}\}$ sont les k voisins les plus proches de x , alors :

$$\hat{y} = \text{mode}(\{y_{i1}, y_{i2}, \dots, y_{ik}\})$$

- y_{ij} est la classe de x_{ij} .
- La fonction **mode** retourne la classe la plus fréquente parmi les k voisins.

Apprentissage paresseux

- Le modèle k-NN n'effectue pas d'apprentissage explicite avant la prédiction.
- Il **stocke** simplement les données d'entraînement et les utilise directement pour prédire.
- On parle d'**apprentissage paresseux** (*lazy learning*).

$$\hat{y} = \text{mode} \left(\frac{y_i}{d(x, x_i)} \text{ pour les } k \text{ plus proches voisins} \right)$$

- $\hat{y}$: classe prédite.
- $d(x, x_i)$: distance entre l'échantillon à prédire et ceux d'entraînement.

Choix du nombre de voisins k

- Le paramètre k influence fortement la performance du modèle :
 - k trop petit $\Rightarrow$ sensible au bruit (surapprentissage).
 - k trop grand $\Rightarrow$ modèle trop général (sous-apprentissage).

$$k^* = \arg \min_k \text{Erreur de validation croisée}(k)$$

- k^* : nombre optimal de voisins choisi par validation croisée.
- Objectif : minimiser l'erreur moyenne sur les données de validation.

Variantes de la méthode des k plus proches voisins

- Il existe plusieurs variantes de la méthode des plus proches voisins (k-NN) :

1. k-NN pondéré

- Chaque voisin est pondéré selon sa distance à l'observation à prédire.
- Pondération inverse possible : les voisins les plus proches ont plus d'influence.

$$\hat{y} = \text{mode} \left(\frac{y_i}{d(x, x_i)} \text{ pour les } k \text{ plus proches voisins} \right)$$

2. k-NN avec distance adaptative

- Les distances sont ajustées selon les caractéristiques locales des données.
- Permet d'améliorer la précision dans les zones à densité variable.

Distances et Similarités

Distances courantes :

- **Euclidienne** : $d(x, y) = \sqrt{\sum (x_i - y_i)^2}$ → Utilisée en reconnaissance d'images (vision par ordinateur).
- **Manhattan** : $d(x, y) = \sum |x_i - y_i|$ → Utilisée pour les grilles urbaines ou les trajectoires GPS.

Similarités :

- **Cosinus** : $\text{sim}(x, y) = \frac{x \cdot y}{\|x\| \|y\|}$ → Utilisée en traitement du langage naturel (similarité de documents).
- **Jaccard** : $\text{sim}(A, B) = \frac{|A \cap B|}{|A \cup B|}$ → Utilisée en recommandation (ensembles d'articles ou de tags).

Distances et Similarités — Exemples de calcul

Distances courantes :

- **Manhattan :**

Exemple : $x = (1, 2)$, $y = (4, 6)$

$$\Rightarrow d = |4 - 1| + |6 - 2| = 7$$

Similarités :

- **Cosinus :**

Exemple : $x = (1, 2, 3)$, $y = (4, 5, 6)$

$$\Rightarrow \text{sim} = \frac{1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6}{\sqrt{14}\sqrt{77}} = 0.968$$

- **Jaccard :**

$$\text{sim}(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Exemple : $A = \{1, 2, 3, 4\}$, $B = \{3, 4, 5, 6\} \Rightarrow \text{sim} = \frac{2}{6} = 0.33$

Similarités entre données catégoriques — Coefficient de Cramér's V

Définition :

$$V = \sqrt{\frac{\chi^2/n}{\min(k_1 - 1, k_2 - 1)}}$$

où :

- χ^2 : statistique du test du chi-carré ;
- n : taille de l'échantillon ;
- k_1, k_2 : nombres de catégories des deux variables.

Exemple : $\chi^2 = 25.6$, $n = 100$, $k_1 = 3$, $k_2 = 4$

Terme de normalisation = $\min(k_1 - 1, k_2 - 1) = \min(2, 3) = 2$

$$V = \sqrt{\frac{25.6/100}{2}} = \sqrt{\frac{0.256}{2}} = \sqrt{0.128} = 0.358$$

→ **Interprétation :** $V = 0.358$ indique une *corrélation modérée* entre les deux variables catégoriques.

Filtrage collaboratif — Principe général et approche utilisateur

Définition :

- Le **filtrage collaboratif** est une technique de recommandation basée sur les préférences des utilisateurs.
- Idée : *“Les utilisateurs qui se ressemblent ont tendance à aimer les mêmes éléments.”*

Deux approches principales :

- 1 Basée sur les **utilisateurs**
- 2 Basée sur les **articles (items)**

Filtrage collaboratif — Principe général et approche utilisateur

Filtrage collaboratif basé sur l'utilisateur

- Recommande des éléments en fonction des préférences d'utilisateurs similaires.
- La prédiction de la préférence de l'utilisateur u pour l'élément i est :

$$\hat{r}_{ui} = \frac{\sum_{v \in N_u} \text{sim}(u, v) r_{vi}}{\sum_{v \in N_u} \text{sim}(u, v)}$$

- N_u : ensemble des utilisateurs similaires à u .
- r_{vi} : note donnée par v à l'élément i .
- $\text{sim}(u, v)$: similarité entre utilisateurs u et v .

Filtrage collaboratif basé sur l'article (item-based)

Principe :

- Recommande des éléments similaires à ceux que l'utilisateur a appréciés dans le passé.
- Utilise la similarité entre les articles eux-mêmes.

Formule de prédiction :

$$\hat{r}_{ui} = \frac{\sum_{j \in N_i} \text{sim}(i, j) r_{uj}}{\sum_{j \in N_i} \text{sim}(i, j)}$$

- N_i : ensemble des articles similaires à i .
- r_{uj} : note de l'utilisateur u pour l'article j .
- $\text{sim}(i, j)$: similarité entre articles i et j (cosinus, Jaccard, etc.).

Applications :

- Recommandation de films (Netflix, IMDb)
- Recommandation de produits (Amazon)
- Suggestion d'amis (réseaux sociaux)

Objectifs

- Montrer, pas à pas, le calcul d'une prédiction par filtrage collaboratif **user-based**.
- Utiliser la similarité de Pearson (centrée) et une combinaison pondérée.
- Fournir tous les calculs pour l'intégrer directement dans un cours.

Jeu de données

On considère 4 utilisateurs $U_1, \dots, U_4$ et 5 items $I_1, \dots, I_5$.

Utilisateur	I_1	I_2	I_3	I_4	I_5
U_1	5	3	4	4	—
U_2	3	1	2	3	3
U_3	4	3	4	3	5
U_4	3	3	1	5	4

Objectif : prédire $r_{1,5}$ (note de U_1 pour

I_5).

Points clés et conclusion

- **Entraînement paresseux** : simple mais coûteux en prédiction.
- **Distance** : élément critique du succès du modèle.
- **Performance** :
 - Efficace en faible dimension.
 - Affecté par le *fléau de la dimension*.
- k-NN = algorithme pédagogique, simple et efficace pour classification, régression et recommandation.